오픈소스 반도체 설계 및 제작 종합설계 6주차 학습노트

이수빈

Gemini가 만든 FIR 필터와 Clock의 RTL을 C++ 기준 모델과 같은 테스트벤치로 비교해 틀린 곳을 찾아 고치고, 5주차의 Space Invaders(Q.14)를 ETRI 0.5um 공정의 표준 셀로 합성, 배치, 배선하여 코어 레이아웃(GDS)까지 만든 과정을 정리한다.

1. 진행 내용

1.1 HLS 서베이 논문

IEEE Access의 HLS 서베이 논문을 읽었다. HLS는 C/C++로 쓴 알고리즘을 RTL로 바꾸는 방법이다. 논문은 도구를 동작 기술 방식, 특정 분야용 언어(DSL) 방식, 데이터플로 방식으로 나누어 비교한다. 어느 방식이든 과정은 할당(allocation), 스케줄링, 바인딩의 세 단계다. 연산에 필요한 자원을 정하고, 각 연산을 어느 클럭에 실행할지 정하고, 연산을 실제 자원에 배정한다. 논문은 포인터, 재귀, 동적 메모리가 있는 일반 C/C++ 코드를 자동으로 옮기는 도구가 아직 없다고 결론 내렸다. 현재 Vitis HLS 문서에서도 재귀와 동적 메모리를 쓰는 C++ 표준 라이브러리는 합성할 수 없다고 안내한다. 한편 최근에는 LLM이 C 코드를 RTL로 바꾸는 연구가 나오고 있고, 이번 과제의 Gemini도 같은 fir8.cpp에서 RTL 다섯 개를 만들었다.

1.2 FIR 8탭 비교 환경

기준 모델 fir8.cpp는 계수 4, 12, 25, 34, 34, 25, 12, 4의 8탭 필터다. 출력은 곱의 합에 64를 더하고 7비트 오른쪽으로 민 값이고, 255를 넘으면 255로 포화시킨다. 64는 반올림 상수이고 RTL에서는 누적기의 시작값으로 들어간다.

Gemini가 만든 RTL은 다섯 구조다.

- 0번 병렬: 곱셈기 8개를 한꺼번에 쓰는 구조
- 1번 직렬: 곱셈기 1개를 8클럭 동안 돌려 쓰는 구조
- 2번 초저면적: 면적을 더 줄이려고 만든 구조
- 3번, 4번: 곱셈기를 쓰지 않는 구조

테스트벤치는 SystemC로 쓰여 있다. 백색 잡음(난수, 진폭의 1/5 범위)에 정현파 네 개를 섞은 입력 4,800개를 만들어 DUT에 넣고, 같은 입력을 C++ 모델에도 넣어 두 출력을 비교한다. 입출력은 핸드셰이크다. out_valid가 0이면 in_valid를 한 번 올려 x를 넣고,

out_valid가 올라올 때까지 기다린 뒤 다음 입력을 넣는다. 같으면 Pass, 다르면 Error를 출력한다. RTL은 Verilator로 C++ 모델로 바꾸어 SystemC에 연결했다. 파이프라인이 있는 구조는 출력이 입력보다 늦게 나오므로 테스트벤치가 기준 값을 OUTPUT_LATENCY만큼 지연시켜 비교한다.

다섯 구조를 한 번에 돌리려고 run_all.sh를 만들었다. 구조마다 fir8.v의 링크를 바꾸고 빌드한 뒤 실행해서 Pass와 Error 개수를 모아 정리했다. 입력의 잡음은 실행할 때마다 달라서, 수정 전에는 같은 구조라도 오류 수가 조금씩 달랐다.

1.3 처음 결과와 원인

처음 돌렸을 때는 4번을 뺀 네 구조에서 4,800개 중 대부분이 틀렸고, 4번은 빌드가 되지 않았다. 원인은 구조마다 달랐다.

0번은 RTL이 틀린 것이 아니라 테스트벤치 설정이 틀렸다. 병렬 구조는 입력에서 출력까지 4번 늦는데 OUTPUT_LATENCY를 1로 둬서 비교 대상이 어긋났다. 4로 두자 4,800개 모두 맞았다. 0번은 4, 나머지는 1로 두어야 했고, 이 값이 구조마다 달라야 한다는 것을 이때 알았다.

1번은 세 가지가 겹쳐 있었다. 첫째, 마지막 탭의 곱이 곱 레지스터에 들어간 다음 클럭에야 누적기에 더해지는데, 출력은 그 전에 나갔다. 계산이 끝난 뒤 한 클럭(calc_flush)을 더 두어 마지막 곱을 누적하게 했다. 둘째, 새 입력이 올 때 곱 레지스터와 누적기를 비우지 않아 앞 샘플의 값이 섞였다. in_valid에서 곱 레지스터는 0, 누적기는 반올림 상수 64로 초기화했다. 셋째, 출력 블록이 end begin으로 쓰여 else가 빠져 있었다. 리셋 분기와 동작 분기가 둘 다 실행되는 구조여서 end else begin으로 고쳤다.

2번은 곱셈기 파이프라인의 지연과 누적 시점이 어긋나 있었다. 계산 중 신호를 두 클럭 늦춘 신호를 만들어 그 신호로 누적하게 하고, out_valid는 계산이 끝난 직후 한 번만 나오게 맞췄다.

3번은 읽기 포인터를 줄이는 시점이 한 칸 어긋나 있었고, 새 입력이 왔을 때 현재 데이터 레지스터가 갱신되지 않았다. 포인터를 bit_cnt가 6일 때 줄이고 in_valid에서 current_data를 x로 받게 했다. 이 구조는 코드 주석 기준으로 입력에서 출력까지 약 66클럭이 걸린다.

4번은 always 블록의 negedge가 megedge로 오타가 나 있어서 Verilator 빌드 자체가 되지 않았다. 한 글자를 고치자 빌드되었고 곧바로 4,800개가 모두 맞았다. 고친 뒤에는 다섯 구조 모두 Pass 4,800, Error 0, "Test Completed Successfully"가 나왔다.

1.4 Clock 예제

Clock은 시, 분, 초를 세는 회로다. C++ 모델(Clock.cpp)과 RTL(clock_rtl.v)의 출력을 매 클럭 비교하는 테스트벤치로 확인했다. 비교 줄은 모두 90,002개이고, 중간(test_count 1,000~1,100) 구간에서는 start를 꺼서 멈춤 동작도 확인한다.

RTL 맨 끝에는 출력을 플립플롭에 한 번 더 담는 블록이 주석으로 남아 있다. 조합 회로의 변화가 바깥으로 나가지 않게 하려는 구조이고, 대신 값이 한 클럭 늦게 나온다. 이 주석을 넣고 빼며 비교했다. 저장소에 있는 RTL은 출력을 assign으로 바로 내보내고, 이 RTL은 90,002줄이 모두 OK였다. 주석을 풀어 출력 레지스터를 둔 Gemini의 처음 RTL로 바꿔서 돌리니 OK 104줄, ERROR 89,898줄이 나왔다. 출력이 C++ 모델보다 한 클럭 늦게 나와서 값은 같아도 매 줄 비교가 어긋난 것이다.

1.5 Q.14 합성에서 코어 GDS까지

5주차의 Q.14를 Qflow의 ETRI 0.5um 설정으로 돌렸다. Q.14 폴더에는 RTL이 번호를 붙인 여러 버전으로 들어 있다(0, 1번은 for 반복문을 쓴 버전, 2~4번은 Gemini가 상태 머신으로 다시 설계한 버전). 이번에는 마지막 번호(4번, v_3)를 합성했다.

합성(yosys)에서 셀 1,760개, 플립플롭 108개가 나왔다. 크기 조정과 버퍼 삽입 뒤에는 셀이 1,838개가 되었고, 이후 배치, 배선, LVS의 소자 수와 같다.

배치(graywolf)는 처음에 그래픽 창이 닫히면서 종료 코드 1로 끝났다. DISPLAY를 비워 창 없이 돌려 보니 종료 코드 2로 아예 실행되지 않았고, 창이 뜬 채로 건드리지 않고 기다리자 끝났다.

타이밍 검사(opensta)에서는 No paths found가 나왔다. 합성 단계가 만든 SDC가 클럭 포트를 clock으로 찾는데 이 설계의 포트는 clk여서, 플립플롭 108개가 모두 클럭이 없는 것으로 처리되었다. SDC를 create_clock -name clk -period 100 [get_ports clk]로 고치자 경로가 잡혔다. 가장 느린 경로는 도착 5.40 ns로 게이트 15단이었고 슬랙은 94.50 ns였다. 배선 전이고 클럭이 이상적인 조건의 값이다.

배선(qrouter)은 1단계에서 306개 넷이 실패했다. 2단계 재시도에서 0이 되었고 3단계를 두 번 더 돌려 최종 라우트 15,109개, 실패 0으로 끝났다. magic으로 옮긴 레이아웃을 LVS로 확인하니 소자 1,838개, 넷 1,845개가 합성 네트리스트와 일치했다.

배선이 끝난 레이아웃에서 적층 비아 검사를 하니 2곳이 나왔다. 이 칩은 금속이 세 층(metal1, metal2, metal3)으로 쌓여 있고, 층 사이는 비아(contact)라는 작은 기둥으로 이어진다. m2contact는 metal1과 metal2를, m3contact는 metal2와 metal3를 잇는다. 적층 비아는 이 두 기둥이 같은 자리에 위아래로 겹쳐서 metal1에서 metal3까지 한 줄로 서 있는 모양이다. 이 공정은 그런 모양을 허용하지 않아서 DRC 규칙 위반으로 본다.
고치는 방법은 위쪽 기둥(m3contact)을 옆으로 옮겨 두 기둥이 겹치지 않게 하는 것이다. magic에서 m3contact만 선택해, 그 위에 연결된 metal3 배선을 따라 8 lambda(2.4 µm) 옮겼다. metal3 배선이 길게 이어져 있어서 옮겨도 위쪽 연결은 그대로 유지된다. 하지만 아래쪽은 끊긴다. 옛 자리에서는 metal2 패드가 m3contact와 닿아 있었는데, 기둥이 옮겨 가자 metal2 패드와 새 자리 사이가 비게 되었다. 그래서 옛 패드에서 새 자리까지 metal2 띠를 그려 이었다. 처음에는 이 띠의 높이를 5 lambda로 그렸다. 그런데 옛 패드의 높이는 7 lambda였다. 띠가 패드보다 낮으니 두 도형이 만나는 곳에 위아래로 홈이 생겼고, 홈 때문에 이웃한 metal2와의 간격이 3 lambda(0.9 µm)보다 좁아졌다. DRC가 Metal2 spacing < 3 (Mosis #9.2) 위반을 낸 이유다. 띠를 패드와 같은 7 lambda 높이로 채우자 홈이 없어지고 위반도 사라졌다. 수정한 뒤 적층 비아 검사, DRC, LVS를 다시 돌려 각각 0, 0, 일치를 확인했다. 연결이 끊기지 않았고 규칙도 지켜졌음을 도구로 확인한 셈이다.

mag2gds로 만든 코어 GDS는 17.4 MB이고 KLayout에서 셀 행과 금속 배선이 보인다. 배선 도구가 만든 적층 비아를 사람이 고쳐야 한다는 것은 연구노트12에 적힌 대로였다.

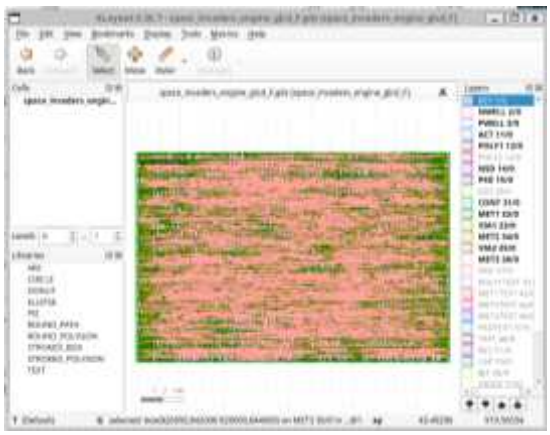


그림 1 KLayout 코어 GDS

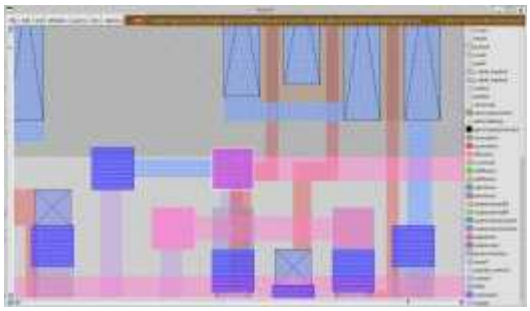


그림 2 적층비아 옮기기 전

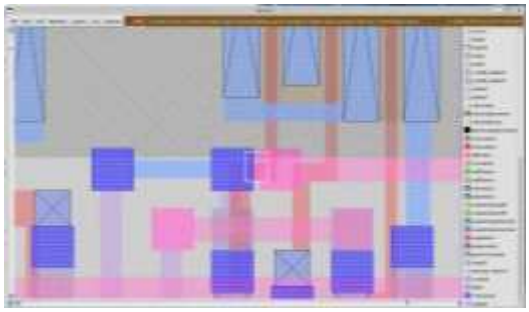


그림 3 적층비아 옮기기 후

2. 결과

2.1 FIR 5구조 수정 전후

수정 후에는 다섯 구조 모두 Pass 4,800이었다.

		수정 전 Error	원인
0	병렬	오류 다수	OUTPUT_LATENCY를 1로 둠(실제 4)
1	직렬		출력 시점, 초기화, else 누락
2	초저면적		계산 신호 지연 불일치
3	곱셈기 x		읽기 포인터 감소 시점
4	곱셈기 x	빌드 실패	negedge 오타

2.2 Q.14 단계별 결과

합성	셀 1,760개(조정 후 1,838개), 플립플롭 108개
LVS	소자 1,838개, 넷 1,845개 일치
적층 비아와 DRC	2곳 수정 후 0 / 오류 0
코어 크기	1,461.6 x 985 µm

2.3 Gemini의 견적과 실제

	Gemini 견적	실제
게이트 수	1,800~2,200	셀 1,760개, 조정 후 1,838개
코어 면적	0.08~0.1 mm²	약 1.44 mm² (1,461.6 x 985 um)

게이트 수 견적은 맞았지만 면적은 약 15배 달랐다. 셀 폭의 합이 30,990 um이고 행

높이가 30 um여서 셀만으로도 약 0.93 mm²다. 칩 탑은 만들지 않았다. 28핀 패드 프레임의 핀 배선 영역이 1,018 x 1,018 um(약 1.04 mm²)인데 셀만 0.93 mm²여서 코어와 배선 여유가 들어가지 않는다. 연구노트12의 FIR PE는 셀 850개에 코어 0.51 mm²로 이 프레임에 들어갔다. 이 프레임에는 셀이 약 1,000개 이하여야 한다.

3. 고찰

3.1 기능이 맞아도 타이밍이 어긋나면 틀린다

이번에 가장 많이 본 오류는 값 자체가 아니라 값이 나오는 시점이었다. Clock은 계산은 같은데 출력 레지스터 하나 때문에 90,002줄 중 89,898줄이 틀렸다. 출력 레지스터는 틀린 설계가 아니고, 출력이 한 클럭 늦다는 사실을 기준 모델과 테스트벤치에 같이 반영해야 한다. FIR 1번과 2번도 마지막 곱을 더하기 전에 출력이 나가거나 out_valid가 한 클럭 어긋난 문제였고, 0번은 RTL이 아니라 지연 설정이 틀렸다. 같은 테스트벤치로 C++ 모델과 RTL을 나란히 비교하니 어긋난 곳이 바로 숫자로 나왔다. 눈으로 코드를 읽어서는 찾기 어려운 종류의 오류였다.

3.2 AI가 만든 RTL은 돌려 봐야 안다

RTL이 처음부터 맞은 것은 0번뿐이었다. 설명과 주석은 그럴듯했지만 나머지 네 구조는 직접 돌려 보니 각각 다른 곳이 틀려 있었다. 1번의 end begin처럼 문법은 통과하지만 동작이 틀린 코드도 있었고, 4번의 megedge처럼 한 글자 오타로 빌드가 막힌 것도 있었다. 테스트벤치를 한 번 만들어 두고 구조 다섯 개를 같은 방식으로 돌리니 어느 쪽이 맞는지 같은 기준으로 판단할 수 있었다. 테스트벤치를 재사용하는 것이 설계를 바꿔 가며 비교하는 데 가장 쓸모가 있었다.

3.3 견적과 실제는 맞는 곳과 틀리는 곳이 갈린다

Gemini가 말한 게이트 수는 실제 합성 결과와 맞았지만 면적은 약 15배 틀렸다. 게이트 수는 로직의 개수에서 나오는 값이라 비슷하게 맞출 수 있어도, 면적은 셀 크기, 행 높이, 배치 밀도, 배선 여유가 곱해져서 정해진다. 셀 1,838개가 1.44 mm²가 된다는 것은 도구를 돌려 보기 전에는 알 수 없었다. 견적은 한 번 도구를 돌려 본 값으로 보정해야 쓸 수 있다고 생각한다.

3.4 칩의 크기가 설계를 정한다

셀이 1,838개인 설계는 28핀 프레임에 들어가지 않았다. 코어의 모든 검증이 통과했는데도

칩으로는 만들 수 없는 설계였다. 5주차에 핀 수가 구조를 정했던 것처럼, 이번에는 프레임의 면적이 구조를 정한다. FIR PE가 850셀에 코어 0.51 mm²였으니, 같은 비율로 1,000셀 안팎이 이 프레임의 한계로 보인다. RTL을 쓰는 단계에서 셀 수의 상한을 알아야 한다고 생각한다.

4. 과제를 수행하며

처음에는 HLS가 C++를 넣으면 RTL이 나오는 도구라고만 생각했다. 직접 해 보니 RTL을 만드는 일보다 그 RTL이 C++와 같은 결과를 내는지 확인하는 일이 훨씬 컸다. 다섯 구조를 모두 맞추는 데 걸린 시간의 대부분이 원인을 찾는 데 쓰였다. 특히 0번은 RTL이 아니라 내가 넣은 지연 값이 틀렸는데, 오류 숫자만 보면 RTL이 틀린 것과 구분되지 않았다. 틀렸다는 결과가 나오면 설계와 테스트벤치 중 어느 쪽이 틀렸는지부터 가려야 한다는 것을 이번에 배웠다.

Q.14는 합성, 배치, 배선, 검증을 도구 순서대로 한 단계씩 따라가며 코어 GDS까지 만들었다. 단계마다 한 번씩은 막혔다. 배치는 창을 닫아서, 타이밍 검사는 SDC의 포트 이름 때문에, 적층 비아는 도구가 만든 배선 때문에 막혔다. 적층 비아를 고칠 때는 이은 띠 높이를 패드보다 낮게 잡아서 DRC 위반을 하나 새로 만들기도 했다. 고친 뒤에 DRC와 LVS를 다시 돌려 확인한 덕분에 바로 잡을 수 있었다. 수정한 뒤에는 검증을 처음부터 다시 돌려야 한다는 것을 몸으로 겪었다.

이번 과제는 하나의 설계가 아이디어에서 칩 레이아웃까지 가는 과정을 처음부터 끝까지 한 바퀴 돌아본 경험이었다. Gemini가 C++ 기준 모델에서 RTL을 만들고, 테스트벤치로 그 RTL이 맞는지 검증했다. 맞는 RTL을 ETRI 0.5um 표준 셀로 합성하고, 배치와 배선을 거쳐 레이아웃을 만들었다. 그 레이아웃을 DRC와 LVS로 다시 검증해 GDS까지 얻었다. 학교 수업에서 따로 배운 논리 회로와 디지털 설계, CMOS와 표준 셀, 설계 규칙이 이 한 흐름 안에서 모두 쓰였다. 각각을 따로 볼 때는 개념이었던 것이, 한 설계가 지나가는 단계로 이어서 보니 서로 어떻게 맞물리는지 보였다.

아쉬운 점은 칩 탑까지 가지 못한 것이다. 코어의 검증은 모두 통과했지만, 셀이 1,838개인 이 설계는 28핀 프레임에 들어가지 않았다. 아직 내 칩으로 무엇을 만들지는 정하지 않았다. 다만 이번처럼 큰 흐름을 한 번 돌면서, 칩을 설계할 때 어느 정도 선에서 시작해야 하는지 가늠할 수 있게 되었다. 면적은 Gemini의 견적과 약 15배 달랐고, 도구는 에러 없이 끝나도 결과가 틀릴 수 있었다. 앞으로 설계를 정할 때는 셀 수의 상한과 핀 수를 먼저 정하고, 합성해서 크기를 확인한 뒤 키워 가려고 한다. 이 과제는 무엇을 만들지보다 얼마만큼을 만들 수 있는지를 알려 준 경험이었다.